Lab 4: Interpreting Classes

Goal: Extend your method interpreter from Lab 3 to support object-oriented programming: class declarations with single inheritance, constructors with a defined initialization order, field access, method invocation with dynamic dispatch, class-type casting, and the `instanceof` operator.

Deadline: 12:00:00 June 29, 2026 (Hard Deadline, No extension)

Programming Language: Java Only.

Different from previous labs, You are expect submit a **Report** when you submit your code.
Please Don't forgot ;)

Task 1: Port Your Lab 3 Implementation

1.1 What Changed in the Grammar

In Lab 4, the compilation unit accepts class declarations alongside method declarations:

```
compilationUnit : (classDeclaration | methodDeclaration | ';')* EOF
```

The program's entry point remains the `int main()` method. All grammar rules for expressions, statements, blocks, control flow, and methods remain compatible with Lab 3. The grammar has been extended with new rules for class declarations, object creation, member access, and `instanceof`.

New keywords:

Keyword	Token	Description
`class`	`CLASS`	Class declaration
`extends`	`EXTENDS`	Single-inheritance clause
`this`	`THIS`	Reference to the current object
`super`	`SUPER`	Reference to the superclass
`instanceof`	`INSTANCEOF`	Runtime class-type test

The project structure is identical to Lab 3:

```
lab4-classes/
├── pom.xml
├── src/main/java/cn/edu/nju/cs/
│   ├── Main.java
│   ├── MiniJavaLexer.g4                # Updated lexer grammar
│   ├── MiniJavaParser.g4               # Updated parser grammar
│   ├── MiniJavaLexer.java              # Generated (do not modify)
│   ├── MiniJavaParser.java             # Generated (do not modify)
│   ├── MiniJavaParserBaseVisitor.java # Generated base visitor to extend
│   └── ...
└── testcase/
    ├── Class-1.mj
    ├── Inherit-1.mj
    └── ...
```

1.2 What You Need to Do

- Copy the `testcase` and `MiniJava*.g4` files from the Lab 4 starter project into your existing Lab 3 project. Do not forget to regenerate the ANTLR sources after updating the grammar files.
- Verify that all Lab 3 test cases continue to pass after porting.

Note: In lab4, we still main function (same as Lab3) as the entry point

---

Task 2: Class Declarations

2.1 Class Declaration

A class declaration has the form:

```
classDeclaration : 'class' identifier ('extends' identifier)? classBody
classBody        : '{' classBodyDeclaration* '}'
classBodyDeclaration : ';' | methodDeclaration | fieldDeclaration |
constructorDeclaration
```

Note 1: A class body may contain fields, methods, constructors, and empty declarations (`;`). Single inheritance is specified with `extends`.

Note 2: Class declarations are registered in the global scope. A class may be used before its declaration appears in the source file.

```
int main() {
    Dog d = new Dog();
    return 0;
}

class Dog extends Animal { }
class Animal { }
```

2.2 Field Declaration

```
fieldDeclaration : typeType variableDeclarator ';'
typeType         : (identifier | primitiveType) ('[' ']')*
```

`typeType` covers primitive types, array types, and class types uniformly. A field may carry an initializer or be left uninitialized.

```
class Point {
    int x;
    int y = 5;
    Point next;     // class-typed field — receives null by default
}
```

2.3 Method Declaration

Methods inside a class body follow the same `methodDeclaration` grammar as top-level methods in Lab 3. A subclass may declare a method with the same name and parameter types as a superclass method, which overrides the superclass implementation.

```
class Animal {
    void speak() { println("..."); }
    string describe(string prefix) { return prefix + "Animal"; }
}
class Dog extends Animal {
    void speak() { println("Woof"); }   // overrides Animal.speak
    // describe is inherited from Animal
}
```

Note 1: Inside a class method, name resolution follows these rules:

1. An unqualified identifier `x` first resolves as a parameter or local variable; if none exists, it is treated as `this.x`.
2. An unqualified call `foo(args...)` is first resolved as `this.foo(args...)`; if no suitable class method is found, it is resolved as a top-level method.

Note 2: The required uses of `this` and `super` are limited to: explicit constructor invocation (`this(...)`, `super(...)`), field access (`this.x`, `super.x`), and method invocation (`this.foo(...)`, `super.foo(...)`). Other uses, such as assigning `this` to a variable, should be treated as an error.

2.4 Constructor Declaration

```
constructorDeclaration : identifier formalParameters block
```

If a class does not explicitly declare any constructor, an implicit default constructor is generated that is semantically equivalent to `C() { }`.

```
class Point {
    int x;
    int y;
    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
    // default constructor will not be generated
    // since one explicit constructor is already declared
}

class Empty {
    // no constructor declared, default Empty() { } is generated
}
```

2.5 Example

The following class illustrates all four kinds of class body members:

```
class Animal {
    string name;              // field without initializer
    int legs = 4;             // field with initializer

    Animal(string name) {     // constructor
        this.name = name;
    }

    void speak() {            // method
        println("...");
    }

    string describe() {
        return name + " has " + legs + " legs";
    }
}

class Dog extends Animal {
    Dog(string name) {
        // explicit superclass constructor invocation
        super(name);
    }

    void speak() {            // override
        println("Woof");
    }
}
```

2.6 What You Need to Do

- Design and implement a representation for class metadata (fields, methods, constructors, and inheritance relationships).
- Implement the `visitxxx` methods related to class declarations so that this metadata is available before interpretation of `main` begins.

Task 3: Class Objects and Constructors

3.1 Object Creation

Object creation uses `new`:

```
'new' identifier '(' expressionList? ')'
```

Class types are reference types. The default value of a class-typed variable is `null`.

```
Animal a1;                // default value: null
Animal a2 = new Animal(); // allocates an object and invokes Animal()
```

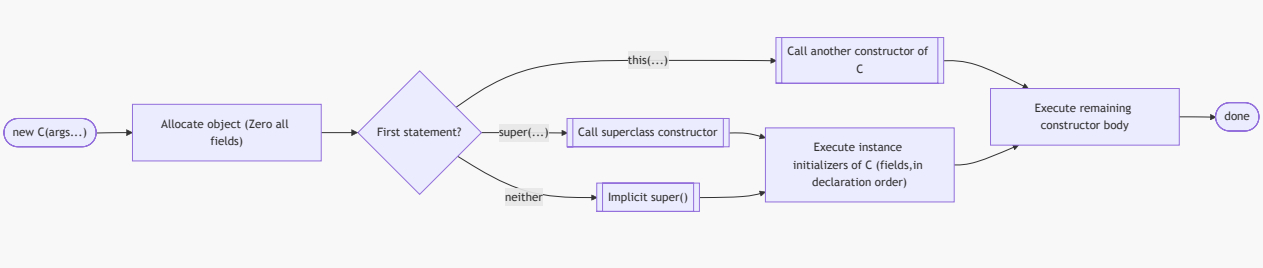
When `new C(args...)` is evaluated, a new object is allocated and all fields across the entire inheritance chain are zeroed to their type defaults before any constructor body executes:

Field type	Default value
`int`	`0`
`char`	`0`
`boolean`	`false`
`string`	`""`
Array or class type	`null`

3.2 Constructor Execution

After allocation, constructor execution can be treated as a structured control flow. The interpreter effectively rewrites constructors into a sequence of:

1. Step 1: constructor chaining (this(...) / super(...))
2. Step 2: superclass initialization
3. Step 3: instance initialization (fields)
4. Step 4: constructor body



Step 1 — constructor chaining (this(...) / super(...))

If the first statement of the constructor is `super(...)` or `this(...)`, an explicit constructor invocation is performed:

- `super(...)` invokes a constructor of the direct superclass.
- `this(...)` invokes another constructor of the current class (constructor delegation).

If the first statement is neither `super(...)` nor `this(...)`, Step 1 is skipped.

Note: `super(...)` and `this(...)` are valid only as the first statement of a constructor body. Any other use is an error.

Step 2 — Superclass initialization

If there is no explicit constructor invocation (`super(...)` or `this(...)`), the default constructor of the direct superclass is invoked automatically.

If the current class has no superclass, skip.

Step 3 — Field initialization

If another constructor of the current class is not invoked via `this(...)` in Step 1, the fields declared in the current class are initialized in declaration order.

For each field that carries an initializer expression, the expression is evaluated and assigned to that field. Fields without an initializer retain the default value established at object allocation.

Step 4 — constructor body

After the preceding stages complete, execution continues with the remaining statements of the constructor body in order.

Examples

Example 1: explicit `super(...)` and field initializers

```
class Animal {
    string name;
    int legs = 4;                          // Animal Step 3: assigned "4"
    Animal(string name) {
        // Step 1 skipped (no super/this)
        // Step 2 skipped (no superclass)
        // Step 3: legs = 4
        this.name = name;                  // Step 4
    }
}
class Dog extends Animal {
    string breed = "unknown";              // Dog Step 3: assigned "unknown"
    int age;                               // Dog Step 3: no initializer
    Dog(string name, int age) {
        super(name);                       // Step 1: invokes Animal(string)
                                           // Step 2 skipped
                                           //      (super(...) already
called)
                                           // Step 3: breed="unknown", age=0
        this.age = age;                    // Step 4
    }
}
// new Dog("Rex", 3):
//   alloc: name="", legs=0, breed="", age=0
//   Dog Step 1 :Animal(string): Animal
//                  → Step 3 legs=4
//                  → Step 4 name="Rex"
//   Dog Step 3: breed="unknown", age=0
//   Dog Step 4: age=3
// result: name="Rex", legs=4, breed="unknown", age=3
```

Example 2: `this(...)` constructor delegation

```
class Dog extends Animal {
    string breed = "unknown";
    int age;
    Dog(string name, int age) {
        super(name);          // Step 1: invokes Animal(string)
        this.age = age;      // Step 4
    }
    Dog(string name) {
        this(name, 0);       // Step 1: delegates to Dog(string, int)
                             // Step 2 skipped (this(...) called)
                             // Step 3 skipped (this(...) called)
    }                        // Step 4: nothing remaining
}
// new Dog("Buddy"):
//   alloc: name="", legs=0, breed="", age=0
//   Dog(string) Step 1 → Dog(string,int):
//     Step 1 → Animal(string): Step 3 legs=4, Step 4 name="Buddy"
//     Step 3: breed="unknown", age=0
//     Step 4: age=0
//   Dog(string) Step 4: nothing
// result: name="Buddy", legs=4, breed="unknown", age=0
```

Example 3: implicit `super()` across a three-level chain

```
class A {
    int x = 1;                           // A Step 3
    // implicit A() {}
}
class B extends A {
    int y = 2;                           // B Step 3
    // implicit B() {}
    // Step 1 skipped, Step 2: implicit A()
}
class C extends B {
    int z = 3;                           // C Step 3
    C() {
        // Step 1 skipped, Step 2: implicit B()
        //   B Step 2: implicit A() → A Step 3 x=1
        //   B Step 3: y=2
        // C Step 3: z=3
    }                                    // Step 4: nothing
}
// new C(): result x=1, y=2, z=3
```

3.3 What You Need to Do

- Implement object creation (`new`), including the zeroing and the four-step constructor execution.
- Implement the `visitxxx` methods related to object creation and constructor invocation.

Task 4: Field Access

4.1 Notation

The following notation is used in Tasks 4 and 5:

- `decl(obj)` — the **declared** type of class object `obj` (from the variable declaration)
- `real(obj)` — the **real** (runtime) type of class object `obj` (from the `new` expression)

```
Animal a = new Dog(); // decl(a) = Animal, real(a) = Dog
```

Note: If `obj` is `null`, any field access `obj.x` triggers a null pointer error (exit code 34).

```
Animal v = null;
var x = v.name; // null pointer error
```

4.2 Access via `this`

The expression `this.x` accesses a field of the current object with the following resolution order:

1. Query the declared type of `this` by `decl(this)`.
2. Starting from `decl(this)`, walk upward through the superclass chain. Return the field `x` from the first class that declares it.

Note: Inside a class method, an unqualified identifier `x` first resolves as a parameter or local variable. If no such variable exists, `x` is treated as `this.x`.

```
class Animal {
    string name = "animal";
    void printName() {
        println(this.name); // decl(this) = Animal → Animal.name → "animal"
        println(name);      // same as this.name
    }
}
class Dog extends Animal {
    string name = "dog";
    void printName() {
        // decl(this) = Dog → Dog.name found first → "dog"
        println(this.name);
    }
}
```

4.3 Access via `super`

The expression `super.x` bypasses the current class and starts field lookup from the direct superclass:

1. Let `decl_class` be the class in which the expression appears.
2. Let `super_class` be the direct superclass of `decl_class`.
3. Starting from `super_class`, walk upward. Return the field `x` from the first class that declares it.

```
class Animal { string name = "animal"; }
class Dog extends Animal {
    string name = "dog";
    void printBoth() {
        println(this.name);  // Dog.name → "dog"
        println(super.name); // starts from Animal → Animal.name → "animal"
    }
}
```

4.4 Access via a Variable

The expression `v.x` resolves field `x` using the *declared* type of `v`:

1. Determine `decl(v)`.
2. Starting from `decl(v)`, walk upward. Return the field `x` from the first class that declares it.

Note: Field access is resolved by the *declared* type, not the real type. This differs from method dispatch (see Task 5).

```
class Animal { string name = "animal"; }
class Dog extends Animal { string name = "dog"; }

Animal v = new Dog();  // decl(v) = Animal, real(v) = Dog
println(v.name);       // decl(v) = Animal → Animal.name → "animal"
```

4.5 What You Need to Do

- Implement the mechanism for class field access.
- Implement null pointer detection for field access on a `null` class object.
- Implement the `visitxxx` methods related to field access expressions (`this.x`, `super.x`, `v.x`).

Task 5: Method Invocation and Dispatch

5.1 Two-Phase Dispatch

Method invocation uses a two-phase model:

1. **Overload resolution** — performed against the *declared* type hierarchy, using the same algorithm (smallest **conversion-count**) as Lab 3. This produces a target method signature `sig`.
2. **Override resolution** — performed against the *real* type. Starting from `real(obj)` and walking upward, the first method whose signature matches `sig` is selected and invoked.

This model implements standard single-dispatch polymorphism (virtual dispatch). In contrast to field access, methods are dispatched via the real type.

Note 1: If `obj` is `null`, any method invocation `obj.foo(...)` triggers a null pointer error (exit code 34).

```
Animal v = null;
v.speak(); // null pointer error
```

```
class Animal {
    void f(Animal a) { System.out.println("Animal-1"); }
    void f(Dog d)    { System.out.println("Animal-2"); }
}

class Dog extends Animal {
    void f(Animal a) { System.out.println("Dog-1"); }
    void f(Dog d)    { System.out.println("Dog-2"); }
}

Animal a = new Dog();
Dog d = new Dog();
a.f(d);   // decl(a) = Animal, real(a) = Dog, decl(d) = Dog, real(d) = Dog
// Phase 1: overload resolution on Animal
//          (candidates: [f(Animal), f(Dog)]) with argument type Dog
//          → sig = f(Dog) since conversion-count is 0
// Phase 2: lookup from Dog → Dog.f(Dog) selected
// result: Dog-2
```

Note 2: A case for conversion-count of class is only calculated wit `decl(obj)`

```
class Animal { void speak() { println("..."); } }
class Dog extends Animal { void speak() { println("Woof"); } }
class Puppy extends Dog { void speak() { println("Yip"); } }

Animal a = new Animal();
Dog    d = new Animal();
Puppy  p = new Animal();

(Animal)a; // conversion-count is 0
(Animal)d; // conversion-count is 1
(Animal)p; // conversion-count is 1 (not a typo)
```

5.2 Invocation via `this`

The call to `this.foo(args...)` dispatches as:

1. Determine `decl(this)`. Collect all candidates named `foo` visible in `decl(this)` and its superclasses. Perform overload resolution to obtain `sig`.
2. Determine `real(this)`. Starting from `real(this)`, walk upward. Select and invoke the first method whose signature matches `sig`.

Note: Inside a class method, an unqualified call `foo(args...)` resolves in order: first as `this.foo(args...)`; if no suitable class method is found, as a top-level method.

```
class Animal {
    void describe() {
        this.speak(); // Phase 1 on decl(this)=Animal
                      // Phase 2 on real(this)
        speak();      // equivalent to this.speak()
    }
    void speak() { println("..."); }
}
class Dog extends Animal {
    void speak() { println("Woof"); }
}

Dog d = new Dog();
d.describe(); // real(this) = Dog → Dog.speak() → output: Woof
```

5.3 Invocation via `super`

The call to `super.foo(args...)` skips the current class and begins lookup from the direct superclass:

1. Let `super_class` be the direct superclass of the enclosing class. Collect candidates named `foo` in `super_class` and its superclasses. Perform overload resolution to obtain `sig`.
2. Starting from `super_class`, walk upward. Select and invoke the first method whose signature matches `sig`.

```
class Animal { void speak() { println("Animal"); } }
class Dog extends Animal {
    void speak() {
        super.speak(); // skips Dog, starts from Animal → Animal.speak()
        println("Dog");
    }
}
// new Dog().speak() outputs:
// Animal
// Dog
```

5.4 Invocation via a Variable

The call `v.foo(args...)` dispatches as:

1. Determine `decl(v)`. Collect candidates named `foo` in `decl(v)` and its superclasses. Perform overload resolution to obtain `sig`.
2. Determine `real(v)`. Starting from `real(v)`, walk upward. Select and invoke the first method whose signature matches `sig`.

```
class Animal { void speak() { println("Animal"); } }
class Dog extends Animal { void speak() { println("Dog"); } }

Animal v = new Dog(); // decl(v) = Animal, real(v) = Dog
v.speak();
// Phase 1: sig = speak() from Animal
// Phase 2: lookup from Dog → Dog.speak()
// output: Dog
```

5.5 What You Need to Do

- Implement the two-phase dispatch mechanism for class method invocation.
- Implement null pointer detection for method invocation on a `null` class object.
- Implement the `visitxxx` methods related to method call expressions on class objects (`this.foo(...)`, `super.foo(...)`, `v.foo(...)`).

Task 6: Type Compatibility

6.1 Implicit Upcast

Assigning a subclass value where a superclass type is expected is implicitly permitted in variable declarations, parameter passing, and return statements.

```
Animal a = new Dog(); // implicit upcast — legal
```

6.2 Explicit Downcast

Converting a class reference to a subclass type uses the cast expression `(C) expr` and requires a runtime check on `real(expr)`:

```
Dog d = (Dog) a; // legal only if real(a) is Dog or a subclass of Dog
```

If `real(a)` is not `Dog` or one of its subclasses at runtime, a type error occurs (exit code 34).

Note: Type casts between array types are not required by this lab.

6.3 Cast Rules

Expression	Condition	Result
`(Super) subExpr`	Always	Legal
`(Sub) superExpr`	`real(expr)` is `Sub` or a subclass of `Sub`	Legal
`(Sub) superExpr`	Otherwise	Type error
`(C) null`	Always	`null`
`(Unrelated) expr`	Types are not in the same inheritance tree	Type error

```
Animal x1 = null;
Animal x2 = new Dog();   // implicit upcast
Dog x3 = (Dog) x2;       // legal — real(x2) is Dog
Animal x4 = new Animal();
Dog x5 = (Dog) x4;       // type error — real(x4) is Animal
```

6.4 Equality on Class Types

The `==` and `!=` operators are supported for class types.

Both operands are class types:

- `decl(obj1)` and `decl(obj2)` must be equal or belong to the same inheritance tree; otherwise a type error occurs.
- The result is `true` if the two references point to the same object; `false` otherwise.

One operand is a class type, the other is not:

- A type error occurs, with the sole exception that comparison with the `null` literal is always legal for any class type.

```
Animal a = new Dog();
Animal b = a;               // b and a reference the same object
Animal c = new Dog();       // c references a different object

boolean r1 = a == b;        // true  – same object
boolean r2 = a == c;        // false – different objects
boolean r3 = a == null;     // legal – false
boolean r4 = null == a;     // legal – false
boolean r5 = a == 42;       // type error
```

6.5 What You Need to Do

- Implement the type-casting mechanism for class types, including the runtime check for downcasts.
- Implement the `visitxxx` methods related to cast expressions.
- Implement `==` and `!=` for class types.

Task 7: `instanceof` Operator

7.1 Syntax and Precedence

```
expression 'instanceof' typeType
```

`instanceof` has the same precedence as relational operators (`<`, `>`, `<=`, `>=`) and produces a `boolean` value.

7.2 Type Constraints

Both operands are subject to the following static checks:

Check	Violation	Result
`decl(obj)` is a class type	e.g. `arr instanceof C` where `arr` is `int[]`	Type error
`TargetType` is a class type	e.g. `obj instanceof int[]`	Type error
`decl(obj)` and `TargetType` belong to the same inheritance tree	e.g. two unrelated classes	Type error

```
class A {}
class B {}

int[] arr = new int[3];
A a = new A();

boolean b1 = arr instanceof A; // type error — decl(arr) is int[]
boolean b2 = a instanceof B;   // type error — A and B are unrelated
```

7.3 Semantics

The result is determined by the real type of the object:

- Returns `true` if `real(obj)` is `TargetType` or a subclass of `TargetType`.
- Returns `false` otherwise.

```
class Animal {}
class Dog extends Animal {}

Animal x1 = new Dog();
Animal x2 = new Animal();

boolean b1 = x1 instanceof Animal; // true  — real(x1) = Dog
                                   //   Dog is a subclass of Animal
boolean b2 = x1 instanceof Dog;    // true  — real(x1) = Dog
boolean b3 = x2 instanceof Animal; // true  — real(x2) = Animal
boolean b4 = x2 instanceof Dog;    // false — real(x2) = Animal
                                   //   not a subclass of Dog
```

7.4 What You Need to Do

- Implement the `instanceof` operator.
- Implement the `visitxxx` methods related to `instanceof` expressions.

Task 8: Input and Output Format

8.1 `print` and `println` for Class Objects

For a class object `x` with declared type `A`:

Condition	Output
`x == null`	`null`
`A` has no suitable `to_string()` method	`real(x)` (the class name)
`A` has a suitable `to_string()` method	result of `x.to_string()`, dispatched via `real(x)`

Note: Any `to_string()` method used for this purpose must have return type `string`.

```
class Animal {
    string to_string() { return "Animal"; }
}
class Dog extends Animal {
    string to_string() { return "Dog"; }
}

Animal a = new Dog(); // decl(a) = Animal, real(a) = Dog
println(a);           // Animal declares to_string();
                      // dispatches to Dog.to_string() → "Dog"

Animal b = null;
println(b);           // null

class Cat extends Animal { } // no to_string() in Cat; Animal also has none

Animal c = new Cat();
println(c);           // no to_string(); outputs real(c) → "Cat"
```

8.2 Normal Output

All output from `print` and `println` goes to `System.out`. When `main` returns normally, the interpreter prints the following line and then calls `System.exit(return_value)` with the return value of `main`:

```
Process exits with <return_value>.
```

```
int main() {
    Animal a = new Dog();
    println(a);
    return 0;
}
```

Output:

```
Dog
Process exits with 0.
```

8.4 Error Output

Error	Exit Code	Output line
Assertion failure	33	`Process exits with 33.`
Null pointer	34	`Process exits with 34.`
Array out of bounds	34	`Process exits with 34.`
Type error	34	`Process exits with 34.`
Other runtime error	34	`Process exits with 34.`

All output, including error lines, goes to `System.out`. `System.err` are allowed to be used for debug output and will not be checked.

Note: Do not throw any class that implements `Error`. Use `Exception` instead. Throwing an `Error` will result in a Wrong Answer.

Task 9: Report

In this section, you are required to complete the following tasks:

1. Explore methods for interpreter performance optimization.
2. Complete your report.

Your Report

The score of this lab will be determined based on the quality of the submitted report. Your report should include the following parts:

- **(Overview)** Performance optimization techniques that could be implemented in your interpreter.
- **(Analysis)** The theoretical basis, code examples, and in-depth analysis for each optimization method.
- **(Conclusion)** A summary of your report.

Your report should be well-organized and clearly explain the motivation, principle, and effects of the optimization techniques (e.g. by providing code examples to demonstrate the effects of the optimization).

Note:

Note 1: Don't be worry, you are only required to complete the report in this task. There is **no need** to actually implement the optimization methods mentioned in your report.

Note 2: But we would happy if you did. ;)

Task 10: Submission

Report Submission

Your report must adhere to the following specifications:

- The report must be submitted in **PDF format**.
- The report should be written in **Chinese or English**.
- The filename must be structured as: **id_name_version.pdf** (e.g., 12345678_张三_1.pdf).

After finishing the report, submit it to NJU Box.

Code Submission

Submit your source files as a zip to our OJ system. OJ will use the Maven files to build the interpreter and test the correctness of the output. Interpreter efficiency is not evaluated in this lab.

The `.zip` should follow the same structure as previous labs:

```
submission.zip
└── outer_folder/
    ├── src/
    └── pom.xml
```